

Chapitre 1

Rappels numériques : erreurs, normes et conditionnement

Introduction

Avant de résoudre le premier système linéaire ou la première équation différentielle de ce module, il faut répondre à une question dérangeante : *un ordinateur ne calcule jamais exactement*. Il travaille avec un nombre fini de chiffres, et chaque addition, chaque division, chaque racine carrée introduit une minuscule erreur d'arrondi. La plupart du temps, ces erreurs sont sans conséquence. Mais parfois – en particulier lorsqu'on approche une structure d'un mécanisme (une ossature presque instable), ou lorsqu'on manipule des matrices de rigidité mal équilibrées – ces erreurs microscopiques peuvent se transformer en erreurs énormes sur le résultat final. Ce chapitre pose les trois outils indispensables pour comprendre et maîtriser ce phénomène : la représentation des nombres en machine, les normes qui permettent de mesurer une erreur, et le *conditionnement*, qui permet de prédire à l'avance si un calcul sera fiable ou dangereux.

🎯 Objectifs pédagogiques

- Comprendre pourquoi un ordinateur ne représente pas exactement les nombres réels (norme IEEE 754).
- Distinguer erreur absolue, erreur relative, erreur d'arrondi et erreur de troncature.
- Savoir calculer et interpréter une norme vectorielle ($\|\cdot\|_1$, $\|\cdot\|_2$, $\|\cdot\|_\infty$) et une norme matricielle induite.
- Définir le conditionnement $\text{cond}(A)$ d'une matrice et l'utiliser pour prédire l'amplification des erreurs dans $Ax = b$.
- Distinguer la stabilité d'un *algorithme* du conditionnement d'un *problème*.
- Mettre en œuvre ces notions en MATLAB (`eps`, `norm`, `cond`).

Prérequis. Algèbre linéaire de base (matrices, vecteurs, valeurs propres), notions de dérivée et de développement de Taylor, bases de la programmation MATLAB (vecteurs, boucles, fonctions).

1.1 Motivation : pourquoi ce problème existe-t-il ?

Un ordinateur stocke un nombre réel sur un nombre fini de bits (32 ou 64 en général). Il ne peut donc représenter qu'un sous-ensemble fini et discret de $\mathbb{R}$. Le nombre $1/3$, par exemple, n'a pas d'écriture binaire finie : il est systématiquement arrondi. Cette erreur, de l'ordre de 10^{-16} en double précision, paraît négligeable – et elle l'est, *tant que le problème traité ne l'amplifie pas*.

Exemple – une structure presque mécanisme

Considérons un treillis formé de deux barres reliant un point P à deux appuis fixes. Si les deux barres sont bien écartées angulairement, une petite variation de la charge appliquée en P produit un petit déplacement. Mais si les deux barres deviennent presque alignées (le treillis se rapproche d'un mécanisme, c'est-à-dire d'une configuration instable), la même petite variation de charge peut produire un déplacement énorme. Le système linéaire $Ku = F$ qui gouverne ce problème est algébriquement identique dans les deux cas – seule la matrice de rigidité K change de *conditionnement*.

Attention

Le résultat numérique d'un calcul peut être totalement faux *sans qu'aucune erreur de programmation n'ait été commise*. C'est le message central de ce chapitre : il faut apprendre à reconnaître les situations où le problème lui-même est intrinsèquement sensible aux erreurs, indépendamment de l'algorithme utilisé pour le résoudre.

1.2 Applications en Génie Mécanique

- **Structures** : matrices de rigidité d'éléments finis mal conditionnées près d'un flambement ou d'une singularité cinématique.
- **Vibrations** : calcul de fréquences propres très proches (modes couplés) – sensibilité numérique accrue.
- **Instrumentation** : lecture de capteurs (jauges de déformation, accéléromètres) avec un nombre fini de chiffres significatifs – propagation de l'erreur de mesure dans un calcul de contrainte.
- **Usinage / CAO** : calculs géométriques (intersections, matrices de changement de repère) sensibles aux erreurs d'arrondi lorsque les configurations sont presque dégénérées.

1.3 Représentation des nombres en machine

1.3.1 Idée intuitive

Un ordinateur ne peut pas stocker une infinité de chiffres. Il choisit donc, pour chaque nombre réel, la valeur représentable la plus proche. On peut imaginer une règle graduée dont les graduations ne sont pas uniformément espacées : très fines près de zéro, de plus en plus grossières lorsque les nombres deviennent grands.

Définition – norme IEEE 754 (double précision)

Un nombre flottant non nul est représenté sous la forme

$$x = \pm 1.b_1b_2 \dots b_{52} \times 2^e, \quad e \in [-1022, 1023],$$

soit 1 bit de signe, 52 bits de mantisse et 11 bits d'exposant (pour une présentation complète de la norme, voir [4]). La précision relative de cette représentation est caractérisée par l'**epsilon machine** $\varepsilon_{\text{mach}}$, défini comme l'écart entre 1 et le plus petit flottant strictement supérieur à 1. En double précision, $\varepsilon_{\text{mach}} = 2^{-52} \approx 2.22 \times 10^{-16}$.

⚠ Attention – $\varepsilon_{\text{mach}}$ et unité d'arrondi

On rencontre parfois la définition « $\varepsilon_{\text{mach}}$ est le plus petit $\varepsilon > 0$ tel que $\text{fl}(1 + \varepsilon) \neq 1$ ». Elle ne désigne *pas* la même quantité : en arrondi au plus proche (mode par défaut de la norme IEEE 754), $\text{fl}(1 + 2^{-53}) = 1$ exactement (règle du « pair le plus proche »), et ce plus petit ε vaut en réalité $2^{-53}(1 + 2^{-52}) \approx 1.11 \times 10^{-16}$, soit la moitié de $\varepsilon_{\text{mach}}$. La quantité $u = \varepsilon_{\text{mach}}/2 = 2^{-53}$ porte le nom d'**unité d'arrondi** : c'est elle qui borne l'erreur relative de représentation (remarque ci-dessous), tandis que $\varepsilon_{\text{mach}}$ mesure l'espacement de la grille des flottants au voisinage de 1.

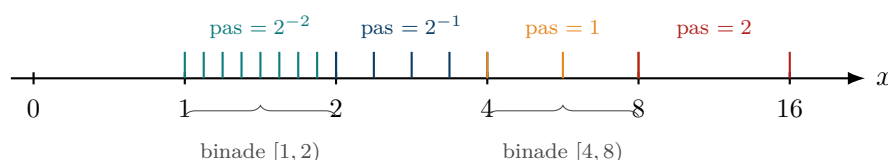


Figure 1.1 – Densité non uniforme des nombres flottants représentables : l'écart entre deux flottants consécutifs double à chaque binade $[2^k, 2^{k+1})$.

i Remarque

Conséquence pratique : l'erreur relative commise en stockant un réel $x \neq 0$ est toujours bornée par $\varepsilon_{\text{mach}}/2$, **quelle que soit la grandeur de x** . C'est pour cela qu'on raisonne presque toujours en erreur *relative* plutôt qu'en erreur absolue en calcul scientifique.

1.4 Erreurs : absolue, relative, arrondi, troncature

☰ Définitions d'erreur

Soit $\tilde{x}$ une valeur approchée d'une quantité exacte $x \neq 0$.

$$e_{\text{abs}} = |\tilde{x} - x| \quad (\text{unités de } x), \quad e_{\text{rel}} = \frac{|\tilde{x} - x|}{|x|} \quad (\text{sans unité}).$$

On distingue deux origines d'erreur, de nature complètement différente :

- **Erreur d'arrondi** (*round-off*) : due à la précision finie de la machine ($\sim \varepsilon_{\text{mach}}$). Elle s'accumule au fil des opérations.
- **Erreur de troncature** (*discretization/truncation*) : due au remplacement d'un objet continu (dérivée, intégrale) par une approximation discrète (différence finie, série tronquée). Elle diminue quand le pas $h \rightarrow 0$.

🔧 Exemple – le compromis classique de la dérivée numérique

La dérivée d'une fonction f peut être approchée par différence finie avant :

$$f'(x_0) \approx \frac{f(x_0 + h) - f(x_0)}{h}.$$

L'erreur de troncature de ce schéma est $O(h)$: elle diminue si h diminue. Mais l'erreur d'arrondi sur la soustraction $f(x_0 + h) - f(x_0)$ (deux nombres très proches) est de l'ordre de $\varepsilon_{\text{mach}}/h$: elle **augmente** si h diminue trop. L'erreur totale est donc $E(h) \approx Ch + \frac{\varepsilon_{\text{mach}}}{h}$, minimale pour $h^* \sim \sqrt{\varepsilon_{\text{mach}}} \approx 1.5 \times 10^{-8}$. Diminuer h en dessous de cette valeur *dégrade* le résultat – un piège classique pour qui pense que « plus petit h = plus précis ».

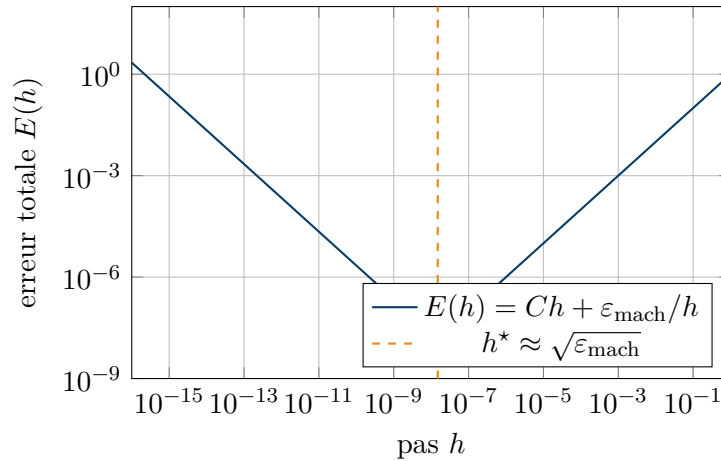


Figure 1.2 – Compromis troncature/arrondi pour la dérivée numérique : l'erreur totale admet un minimum, elle ne décroît pas indéfiniment.

1.5 Normes vectorielles et matricielles

Pour parler d'« erreur » sur un vecteur (un champ de déplacements, un résidu de forces) ou sur une matrice, il faut une mesure scalaire de leur « taille ». C'est le rôle d'une norme.

☰ Définitions usuelles sur $\mathbb{R}^n$

Pour $x = (x_1, \dots, x_n) \in \mathbb{R}^n$:

$$\|x\|_1 = \sum_{i=1}^n |x_i|, \quad \|x\|_2 = \left(\sum_{i=1}^n x_i^2 \right)^{1/2}, \quad \|x\|_\infty = \max_{1 \leq i \leq n} |x_i|.$$

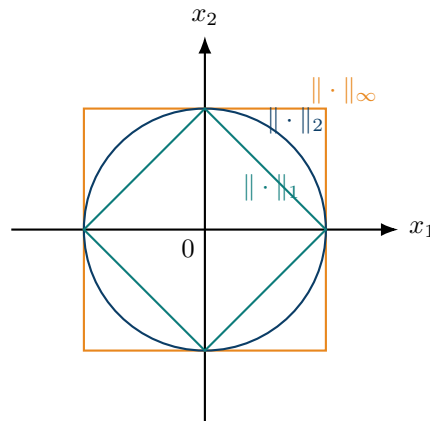


Figure 1.3 – Boules unités $\{x : \|x\| = 1\}$ dans $\mathbb{R}^2$ pour les normes $\|\cdot\|_1$, $\|\cdot\|_2$ et $\|\cdot\|_\infty$.

☰ Définition – norme matricielle induite

À toute norme vectorielle $\|\cdot\|$ on associe une norme matricielle *induite* (ou subordonnée)

$$\|A\| = \max_{x \neq 0} \frac{\|Ax\|}{\|x\|}.$$

Elle mesure le facteur d'amplification maximal que A peut faire subir à un vecteur.

i Remarque

En pratique, on utilise surtout $\|A\|_2$ (liée aux valeurs singulières de A , ou aux valeurs propres si A est symétrique) et $\|A\|_\infty = \max_i \sum_j |a_{ij}|$ (somme maximale des valeurs absolues par ligne, très simple à calculer à la main).

1.6 Conditionnement d'un problème

1.6.1 Principe mathématique

On cherche à répondre à la question suivante : si les données d'un problème (par exemple le second membre b dans $Ax = b$) sont connues avec une petite erreur δb , quelle erreur δx cela induit-il sur la solution ?

▤ Théorème (sensibilité de $Ax = b$)

Soit A inversible, $Ax = b$ et $A(x + \delta x) = b + \delta b$. Alors

$$\frac{\|\delta x\|}{\|x\|} \leq \text{cond}(A) \frac{\|\delta b\|}{\|b\|}, \quad \text{avec} \quad \text{cond}(A) = \|A\| \|A^{-1}\|.$$

Démonstration. De $A\delta x = \delta b$, on tire $\delta x = A^{-1}\delta b$, donc $\|\delta x\| \leq \|A^{-1}\| \|\delta b\|$. De $Ax = b$, on tire $\|b\| \leq \|A\| \|x\|$, donc $\frac{1}{\|x\|} \leq \frac{\|A\|}{\|b\|}$. En multipliant ces deux inégalités :

$$\frac{\|\delta x\|}{\|x\|} \leq \|A^{-1}\| \|\delta b\| \cdot \frac{\|A\|}{\|b\|} = \text{cond}(A) \frac{\|\delta b\|}{\|b\|}. \quad \square$$

i Remarque

On a toujours $\text{cond}(A) \geq 1$. Un conditionnement proche de 1 signifie que le problème est *bien posé numériquement* : une petite erreur relative sur les données reste petite sur la solution. Un conditionnement élevé (10^6 , 10^{10} ...) signifie que le problème est *intrinsèquement sensible* : même l'algorithme le plus stable ne pourra pas garantir un résultat fiable si les données sont imprécises.

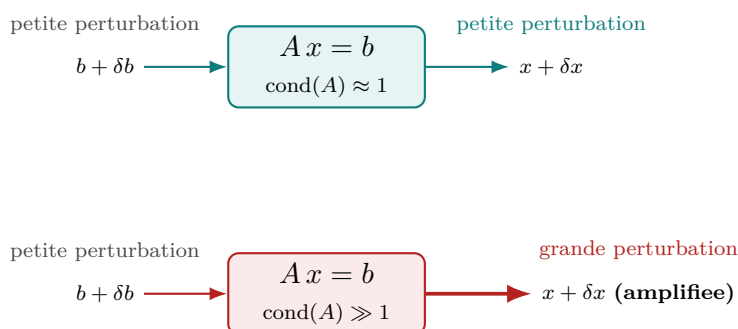


Figure 1.4 – Propagation d'une même perturbation relative δb dans un système bien conditionné (haut) et mal conditionné (bas).

⚗ Exemple mécanique – deux ressorts

Deux ressorts de raideurs k_1 et k_2 montés en série entre deux points fixes, avec un nœud intermédiaire de déplacement inconnu u . L'équilibre s'écrit $Ku = F$ avec $K = k_1 + k_2$

(scalaire ici). Si $k_1 \gg k_2$ (un ressort très rigide, un ressort très souple), le système reste bien conditionné (cas scalaire, $\text{cond} = 1$). En revanche, pour un treillis 2D dont deux barres deviennent quasi-colinéaires, la matrice de rigidité 2×2 associée au nœud libre voit son plus petit rapport de valeurs propres s'effondrer – $\text{cond}_2(K)$ explose, et une petite incertitude sur le chargement F (ex. vent, vibrations) produit un déplacement prédit totalement incertain.

1.7 Stabilité numérique d'un algorithme

Définition

Le **conditionnement** est une propriété du *problème* (indépendante de la méthode de résolution). La **stabilité** est une propriété de l'*algorithme* : un algorithme est dit stable s'il n'amplifie pas artificiellement les erreurs d'arrondi au-delà de ce que le conditionnement du problème laisse prévoir.

Attention classique

Un bon algorithme appliqué à un problème mal conditionné donnera toujours un mauvais résultat : ce n'est pas un bug, c'est une limite intrinsèque de l'information disponible. À l'inverse, un mauvais algorithme (instable) peut dégrader un problème parfaitement bien conditionné. Il faut donc toujours distinguer les deux causes avant de « déboguer » un résultat numérique suspect.

1.8 Algorithme et organigramme : calcul de l'épsilon machine

Algorithme 1: Estimation expérimentale de $\varepsilon_{\text{mach}}$

Donnees: aucune

Resultat: ε , approximation de $\varepsilon_{\text{mach}}$

```

1  $\varepsilon \leftarrow 1$ ;
2 tant que  $\text{fl}(1 + \varepsilon/2) > 1$  faire
3    $\varepsilon \leftarrow \varepsilon/2$ ;
4 retourner  $\varepsilon$ ;
```

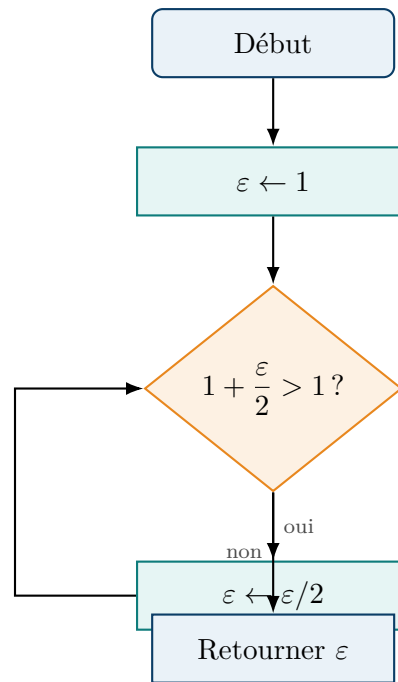


Figure 1.5 – Organigramme de l’algorithme de calcul itératif de l’epsilon machine.

1.9 Implémentation MATLAB



MATLAB – epsilon machine, normes et conditionnement

```

1  %% chap01_erreurs_conditionnement.m
2  % Illustration : epsilon machine, normes, conditionnement d'une
   matrice
3  clear; close all; clc;
4
5  %% 1) Epsilon machine (calcul "a la main" vs valeur MATLAB)
6  eps_calc = 1;
7  while (1 + eps_calc/2) > 1
8      eps_calc = eps_calc/2;
9  end
10 fprintf('epsilon machine calcule : %.4e\n', eps_calc);
11 fprintf('epsilon machine (MATLAB) : %.4e\n', eps);
12
13 %% 2) Compromis troncature / arrondi sur la derivee de sin(x) en x0
14 x0 = 1; h = 10.^(-1:-1:-16);
15 df_exact = cos(x0);
16 df_approx = (sin(x0+h) - sin(x0)) ./ h;
17 erreur = abs(df_approx - df_exact);
18
19 figure;
20 loglog(h, erreur, 'o-', 'Color', [0.043 0.239 0.400], 'LineWidth', 1.5);
21 xlabel('pas h'); ylabel('erreur absolue');
22 title('Compromis troncature/arrondi -- derivee de sin(x)');
23 grid on;
24
25 %% 3) Normes d'un vecteur
26 x = [3; -4; 1];
27 fprintf('||x||_1 = %g\n', norm(x,1));
  
```

```

28 fprintf('||x||_2 = %g\n', norm(x,2));
29 fprintf('||x||_inf = %g\n', norm(x,Inf));
30
31 %% 4) Conditionnement d'une matrice de rigidite quasi-singuliere
32 theta = deg2rad(5); % angle entre les deux barres (presque
    colineaires)
33 K = [1 cos(theta); 0 sin(theta)];
34 fprintf('cond_2(K) pour theta = 5 deg : %.3e\n', cond(K));
35
36 theta2 = deg2rad(45);
37 K2 = [1 cos(theta2); 0 sin(theta2)];
38 fprintf('cond_2(K) pour theta = 45 deg : %.3e\n', cond(K2));

```

1.9.1 Interprétation physique des résultats

La courbe log-log de la partie 2 présente la forme en « V » caractéristique annoncée à la figure 1.2 : l'erreur diminue puis remonte lorsque h devient trop petit. La partie 4 montre numériquement ce que l'exemple du treillis annonce qualitativement : faire tendre l'angle entre les deux barres vers 0° (barres quasi-colinéaires, treillis proche d'un mécanisme) fait exploser $\text{cond}_2(K)$, alors qu'un angle de 45° donne un conditionnement modéré.

Avantages, limites et pièges

- **Avantage** : le conditionnement se calcule *a priori*, avant même de résoudre le système – il permet d'anticiper une perte de fiabilité.
- **Limite** : $\text{cond}(A)$ ne donne qu'une borne *supérieure* de l'amplification d'erreur ; le comportement réel peut être bien meilleur pour un second membre b particulier.
- **Piège** : calculer $\text{cond}(A)$ via A^{-1} explicitement est lui-même coûteux et parfois instable pour de grandes matrices ; MATLAB utilise en interne les valeurs singulières ($\text{cond} =$ rapport de la plus grande à la plus petite valeur singulière), jamais une inversion naïve.

1.10 Exercices résolus

Les cinq exercices qui suivent reprennent *un par un* les objectifs annoncés en ouverture du chapitre : l'épsilon machine, l'erreur de représentation, la lecture des normes, le calcul d'un conditionnement, et enfin l'usage du conditionnement pour *prédire* l'erreur avant de la subir. Chaque résultat est reproductible en MATLAB avec les commandes rappelées en fin d'énoncé.

✂ Exercice résolu – L'épsilon machine, à la main puis en machine

Énoncé. En double précision IEEE 754, la mantisse compte 52 bits fractionnaires.

1. Donner $\varepsilon_{\text{mach}}$ et l'unité d'arrondi u .
2. Expliquer pourquoi, en machine, $1 + \varepsilon_{\text{mach}}/2$ vaut *exactement* 1, alors que $1 + \varepsilon_{\text{mach}} > 1$.
3. Combien de divisions la boucle « tant que $1 + e/2 > 1$, faire $e \leftarrow e/2$ » effectue-t-elle en partant de $e = 1$?

Solution.

1. L'épsilon machine est l'écart entre 1 et le flottant qui le suit immédiatement :

$$\varepsilon_{\text{mach}} = 2^{-52} \approx 2,2204 \times 10^{-16}, \quad u = \frac{\varepsilon_{\text{mach}}}{2} = 2^{-53} \approx 1,1102 \times 10^{-16}.$$

Ces deux nombres ne disent pas la même chose : $\varepsilon_{\text{mach}}$ est le *pas de la grille* des flottants au

voisinage de 1, tandis que u borne l'erreur relative commise en rangeant un réel quelconque en machine (au pire la moitié d'un pas).

2. Autour de 1, les flottants sont espacés de 2^{-52} : le voisin de droite est $1 + 2^{-52}$, et il n'y a *rien* entre les deux. Le nombre exact $1 + 2^{-53}$ tombe donc précisément au milieu de l'intervalle $[1, 1 + 2^{-52}]$. L'arrondi au plus proche doit alors départager, et la règle IEEE 754 « au pair » retient celui des deux voisins dont le dernier bit de mantisse est nul, c'est-à-dire 1. D'où :

$$1 + \frac{1}{2}\varepsilon_{\text{mach}} = 1 \quad \text{en machine,} \quad \text{alors que} \quad 1 + \varepsilon_{\text{mach}} > 1$$

puisque cette dernière valeur, elle, est exactement représentable. La frontière n'est pas la moitié mais le *milieu* : dès que l'on dépasse strictement u , par exemple avec $1 + 0,75\varepsilon_{\text{mach}}$, l'arrondi bascule vers le voisin de droite et le résultat redevient > 1 .

3. En partant de $e = 1 = 2^0$, on a $e = 2^{-k}$ après k divisions. La boucle s'arrête dès que $1 + e/2$ n'est plus distinguable de 1, c'est-à-dire dès que $e/2 \leq u = 2^{-53}$, soit $e \leq 2^{-52}$. Elle effectue donc exactement **52 divisions** et rend $e = 2^{-52} = \varepsilon_{\text{mach}}$: l'algorithme du § 1.8 *mesure* bien la longueur de la mantisse, sans qu'on ait eu besoin de la lui donner.

Vérification MATLAB. `eps → 2.2204e-16; 1+eps/2 == 1 → 1 (vrai); 1+eps > 1 → 1 (vrai).`

✂ Exercice résolu – Erreur relative de représentation

Énoncé. On représente $x = 0.1$ en base 2 avec une mantisse de 4 bits (arrondi au plus proche). Sachant que $0.1_{10} = 0.0001100110011\dots_2$ (motif périodique), évaluer l'erreur relative commise.

Solution. On commence par **normaliser** l'écriture binaire, c'est-à-dire la ramener à la forme $1.b_1b_2\dots \times 2^e$:

$$0.1_{10} = 0.0001100110011\dots_2 = 1.1001100110011\dots_2 \times 2^{-4}.$$

Il faut maintenant conserver 4 bits après le premier bit significatif (implicite), soit la mantisse $1.\underbrace{1001}_{4 \text{ bits}}|10011\dots$. Le premier bit écarté vaut 1 et le reste n'est pas nul : l'arrondi au plus proche se fait donc **vers le haut**,

$$1.1001_2 + 2^{-4} = 1.1010_2 = 1 + \frac{1}{2} + \frac{1}{8} = 1.625,$$

d'où $x \approx 1.625 \times 2^{-4} = 0.1015625$ et

$$e_{\text{rel}} = \frac{|0.1015625 - 0.1|}{0.1} = \frac{0.0015625}{0.1} = 1.5625 \times 10^{-2}.$$

Avec seulement 4 bits, l'erreur relative est donc de 1,5625% – à comparer aux $\sim 10^{-16}$ obtenus avec les 52 bits de la double précision IEEE 754. Ceci illustre concrètement pourquoi la taille de la mantisse contrôle directement la précision relative, indépendamment de l'exposant.

Remarque – arrondi contre troncature, à nombre de bits égal. Comparons les deux modes sur la *même* mantisse de 4 bits. Tronquer, c'est garder $1.1001_2 = 1.5625$ sans rien ajouter, d'où

$$x \approx 1.5625 \times 2^{-4} = 0.09765625, \quad e_{\text{rel}} = \frac{0.00234375}{0.1} = 2,34375 \times 10^{-2},$$

soit 2,34% contre 1,5625% pour l'arrondi : ici, la troncature coûte un facteur **1,5**.

Pourquoi 1,5 et non 2 ? Tout se lit sur les bits écartés. Après 1.1001 , la queue vaut $0.100110011\dots_2 = 0,6$ *unité du dernier bit* (ulp). La troncature commet donc une erreur de

0,6 ulp, tandis que l'arrondi, qui remonte au flottant supérieur, ne commet que $1 - 0,6 = 0,4$ ulp – d'où le rapport $0,6/0,4 = 1,5$, propre à ce nombre. Dans le *pire* cas en revanche, la troncature approche 1 ulp alors que l'arrondi ne dépasse jamais $\frac{1}{2}$ ulp : l'erreur maximale est bien divisée par **deux**. C'est pour cette raison, et parce que l'erreur devient centrée (donc sans biais systématique lorsqu'elle s'accumule sur de nombreuses opérations), que la norme IEEE 754 impose l'arrondi au plus proche comme mode par défaut.

✂ Exercice résolu – Lire les trois normes, sur un vecteur puis sur une matrice

Énoncé. Soit le vecteur des déplacements nodaux $x = (3, -4, 12)^T$ (en mm) et la matrice

$$A = \begin{pmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{pmatrix},$$

qui est la matrice tridiagonale que l'on retrouvera à chaque discrétisation de ce cours.

1. Calculer $\|x\|_1$, $\|x\|_2$, $\|x\|_\infty$ et vérifier l'encadrement attendu.
2. Calculer $\|A\|_1$ et $\|A\|_\infty$. Que remarque-t-on, et pourquoi ?
3. Donner $\|A\|_2$.

Solution.

1. Les trois normes se lisent directement sur les composantes :

$$\|x\|_1 = 3 + 4 + 12 = 19, \quad \|x\|_2 = \sqrt{9 + 16 + 144} = \sqrt{169} = 13,$$

$$\|x\|_\infty = \max(3, 4, 12) = 12.$$

L'encadrement général $\|x\|_\infty \leq \|x\|_2 \leq \|x\|_1$ est bien vérifié : $12 \leq 13 \leq 19$. Ce n'est pas une coïncidence numérique mais la traduction du fait que la boule unité de $\|\cdot\|_1$ (le losange) est incluse dans celle de $\|\cdot\|_2$ (le cercle), elle-même incluse dans celle de $\|\cdot\|_\infty$ (le carré).

Lecture mécanique. Les trois nombres répondent à trois questions différentes sur le même champ de déplacement : $\|x\|_\infty = 12$ mm est le déplacement du nœud le plus sollicité – c'est lui qu'on compare à une tolérance ; $\|x\|_1 = 19$ mm cumule tous les déplacements ; $\|x\|_2 = 13$ mm est le compromis usuel, celui qui apparaît naturellement dans les bilans d'énergie.

2. Les normes matricielles *induites* par $\|\cdot\|_1$ et $\|\cdot\|_\infty$ se lisent, elles aussi, sans calcul lourd : la première est le maximum des sommes de valeurs absolues *par colonne*, la seconde le maximum *par ligne*. Ici les sommes valent 3, 4 et 3 dans les deux sens, d'où

$$\|A\|_1 = \|A\|_\infty = 4.$$

L'égalité n'a rien de fortuit : $\|A\|_1 = \|A^T\|_\infty$ pour toute matrice, et A est ici symétrique.

3. A étant symétrique, sa norme 2 est le rayon spectral, c'est-à-dire la plus grande valeur propre en module. Les valeurs propres de la tridiagonale $(-1, 2, -1)$ d'ordre n sont connues : $\lambda_k = 2 - 2 \cos(k\pi/(n+1))$. Pour $n = 3$:

$$\lambda_1 = 2 - \sqrt{2} \approx 0,586, \quad \lambda_2 = 2, \quad \lambda_3 = 2 + \sqrt{2} \approx 3,414,$$

d'où $\|A\|_2 = 2 + \sqrt{2} \approx 3,414$. On retiendra que $\|A\|_2 \leq \|A\|_\infty$ ici, mais que ces trois normes matricielles restent du même ordre de grandeur : le choix de la norme change les chiffres, pas les conclusions.

Vérification MATLAB. `norm(x,1)`, `norm(x,2)`, `norm(x,inf)` et, pour la matrice, `norm(A,1)`, `norm(A,inf)`, `norm(A,2)`.

✂ Exercice résolu – Conditionnement d’une matrice de rigidité

Énoncé. Un nœud de treillis relié par deux barres est gouverné par $Ku = F$ avec

$$K = \begin{pmatrix} 1 & 1 \\ 1 & 1.0001 \end{pmatrix}.$$

Calculer $\text{cond}_\infty(K)$ et commenter.

Solution. $\det K = 1 \times 1.0001 - 1 \times 1 = 0.0001$. L’inverse est

$$K^{-1} = \frac{1}{0.0001} \begin{pmatrix} 1.0001 & -1 \\ -1 & 1 \end{pmatrix} = \begin{pmatrix} 10001 & -10000 \\ -10000 & 10000 \end{pmatrix}.$$

Norme ∞ (somme max des valeurs absolues par ligne) : $\|K\|_\infty = \max(2, 2.0001) = 2.0001$, et $\|K^{-1}\|_\infty = \max(20001, 20000) = 20001$. Donc

$$\text{cond}_\infty(K) = \|K\|_\infty \|K^{-1}\|_\infty \approx 2.0001 \times 20001 \approx 4.0 \times 10^4.$$

Un conditionnement de l’ordre de 10^4 signifie qu’une erreur relative de 10^{-4} sur le chargement F (par exemple une incertitude de mesure) peut se traduire par une erreur relative proche de 100% sur le déplacement calculé u : la solution numérique n’est plus fiable sans données beaucoup plus précises.

✂ Exercice résolu – Des normes au conditionnement : prédire l’erreur avant de la subir

Énoncé. On reprend la matrice A de l’exercice sur les normes et l’on résout $Ax = b$ avec $b = (1, 0, 1)^\top$. La première composante de b provient d’un capteur, connu à 1% près.

1. Calculer A^{-1} puis $\text{cond}_\infty(A)$.
2. Majorer l’erreur relative commise sur x .
3. Comparer cette borne à l’erreur *réellement* commise pour $\delta b = (0,01, 0, 0)^\top$.

Solution.

1. On a $\det A = 4$ et

$$A^{-1} = \frac{1}{4} \begin{pmatrix} 3 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 3 \end{pmatrix}.$$

Les sommes de lignes de A^{-1} valent $6/4$, $8/4$ et $6/4$, donc $\|A^{-1}\|_\infty = 2$. Avec $\|A\|_\infty = 4$ calculé précédemment :

$$\text{cond}_\infty(A) = \|A\|_\infty \|A^{-1}\|_\infty = 4 \times 2 = 8.$$

2. Le théorème de sensibilité donne directement, *sans résoudre le système perturbé* :

$$\frac{\|\delta x\|_\infty}{\|x\|_\infty} \leq \text{cond}_\infty(A) \frac{\|\delta b\|_\infty}{\|b\|_\infty} = 8 \times 1\% = 8\%.$$

3. Vérifions. La solution non perturbée est $x = A^{-1}b = (1, 1, 1)^\top$. La perturbation se propage linéairement :

$$\delta x = A^{-1} \delta b = \frac{0,01}{4} (3, 2, 1)^\top = (0,0075, 0,0050, 0,0025)^\top,$$

d’où $\|\delta x\|_\infty / \|x\|_\infty = 0,0075 = 0,75\%$, très en deçà des 8% garantis.

	erreur sur b	erreur sur x
borne du théorème	1%	$\leq 8\%$
mesure effective	1%	0,75%

Comment lire ce résultat. La borne n'est pas une prédiction, c'est une *garantie de pire cas* : elle vaut pour *toutes* les perturbations de même taille, et n'est atteinte que si δb pointe dans la direction la plus amplifiée par A^{-1} . Ici δb n'y pointe pas, d'où le facteur 10 de marge. L'intérêt pratique est ailleurs : avec $\text{cond}_{\infty}(A) = 8$, on sait *avant* de lancer le calcul qu'un capteur à 1% ne peut pas ruiner le résultat – à comparer au treillis presque aligné de l'exercice précédent, où $\text{cond} \approx 4 \times 10^4$ interdit toute confiance sans une instrumentation bien plus fine.

Vérification MATLAB. `cond(A,inf) → 8`; l'opérateur `A\b` pour les deux seconds membres, puis comparaison des `norm(...,inf)`.

Résumé du chapitre

Résumé du chapitre

- Un ordinateur représente les réels avec une précision relative finie $\varepsilon_{\text{mach}} \approx 2.2 \times 10^{-16}$ (double précision).
- Erreur d'arrondi (liée à $\varepsilon_{\text{mach}}$) et erreur de troncature (liée au pas de discrétisation h) ont des comportements opposés : leur somme admet un minimum, pas une décroissance indéfinie.
- Les normes ($\|\cdot\|_1, \|\cdot\|_2, \|\cdot\|_{\infty}$) fournissent une mesure scalaire de la taille d'un vecteur ou d'une erreur.
- Le conditionnement $\text{cond}(A) = \|A\| \|A^{-1}\|$ mesure l'amplification maximale d'une erreur relative sur les données d'un système $Ax = b$ – c'est une propriété du *problème*, pas de l'algorithme.
- La stabilité d'un *algorithme* est une notion distincte : un algorithme stable n'amplifie pas les erreurs au-delà de ce que le conditionnement prévoit déjà.

Carte mentale

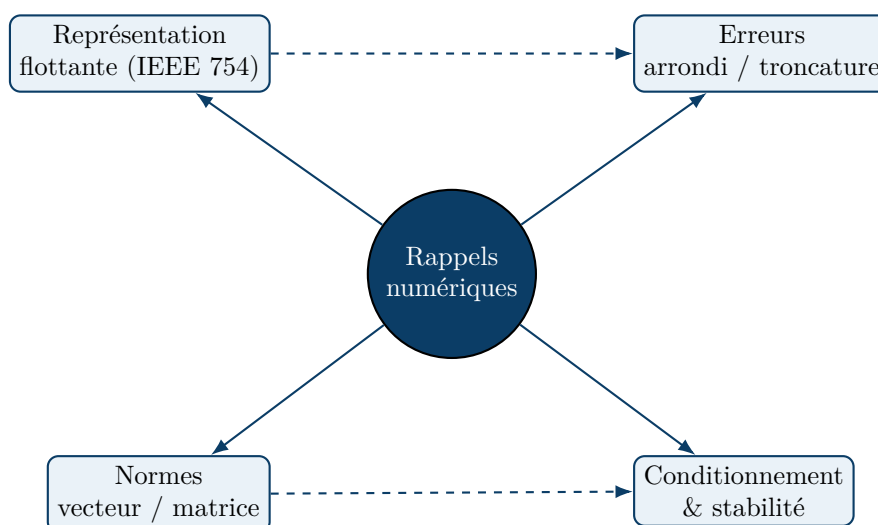


Figure 1.6 – Carte mentale du chapitre 1.

☰ Formulaire

$$\varepsilon_{\text{mach}} \approx 2.22 \times 10^{-16} \text{ (double précision)}$$

$$e_{\text{rel}} = \frac{|\tilde{x} - x|}{|x|}$$

$$\|x\|_1 = \sum_i |x_i|, \quad \|x\|_2 = \sqrt{\sum_i x_i^2}, \quad \|x\|_\infty = \max_i |x_i| \quad \|A\|_\infty = \max_i \sum_j |a_{ij}|$$

$$\text{cond}(A) = \|A\| \|A^{-1}\| \geq 1$$

$$\frac{\|\delta x\|}{\|x\|} \leq \text{cond}(A) \frac{\|\delta b\|}{\|b\|}$$

🔍 Quiz

1. L'epsilon machine représente : (a) la plus petite valeur représentable, (b) la plus petite erreur relative significative, (c) le nombre de bits de l'exposant.
2. Si $\text{cond}(A) = 10^8$ et que les données sont connues à 10^{-6} près, la solution peut être faussée à : (a) 10^{-14} , (b) 10^2 (soit totalement faussée), (c) 10^{-6} près exactement.
3. Réduire indéfiniment le pas h dans une différence finie : (a) améliore toujours la précision, (b) finit par dégrader la précision à cause de l'arrondi, (c) n'a aucun effet.
4. Le conditionnement d'une matrice est une propriété : (a) de l'algorithme utilisé, (b) du problème lui-même, (c) du compilateur.

Réponses : 1b - 2b - 3b - 4b.

Exercices d'entraînement

1. **[Facile]** Calculer $\|x\|_1$, $\|x\|_2$ et $\|x\|_\infty$ pour $x = (2, -6, 3)$.
2. **[Facile]** Deux mesures d'une longueur donnent $\tilde{L} = 100.4$ mm pour une valeur exacte $L = 100$ mm. Calculer l'erreur absolue et l'erreur relative.
3. **[Moyen]** Montrer que pour toute matrice A inversible, $\text{cond}(A) = \text{cond}(A^{-1})$.
4. **[Moyen]** Calculer $\text{cond}_\infty(A)$ pour $A = \begin{pmatrix} 4 & 0 \\ 0 & 2 \end{pmatrix}$ et interpréter physiquement (deux ressorts indépendants de raideurs 4 et 2).
5. **[Difficile]** Pour la matrice $K(\theta) = \begin{pmatrix} 1 & \cos \theta \\ 0 & \sin \theta \end{pmatrix}$ du treillis à deux barres, tracer (à la main ou en MATLAB) $\text{cond}_2(K)$ en fonction de $\theta \in]0^\circ, 90^\circ]$ et identifier la zone d'angles à éviter en conception.
6. **[Très difficile]** On mesure un vecteur force $F = (F_1, F_2)$ avec une incertitude relative de 1% sur chaque composante. En utilisant le théorème de sensibilité, donner une borne sur l'incertitude relative du déplacement u solution de $Ku = F$ pour la matrice de la question précédente avec $\theta = 5^\circ$. Commenter la pertinence physique d'une telle configuration de treillis.

💡 Astuce

Les corrigés détaillés de ces exercices d'entraînement se trouvent dans le document séparé `corriges/corrige_chap01.tex`, conformément à la consigne de ne jamais mélanger énoncés et solutions dans le même support de cours.